

マケデコセミナー

投資戦略を量産せよ！！

2025/7/3 19:30-



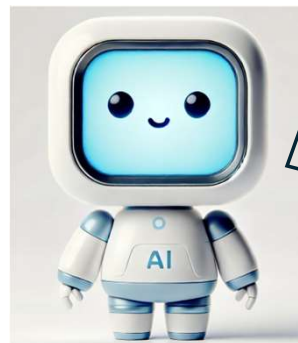
株価予測の最強アーキテクチャはどれだ？AIエージェントによる探索
UKI@blog_uki

AIエージェントと投資戦略 1/2

(1) 広義で「投資戦略を考えて」

- ・ 一昔前まではほぼ不可能だった
- ・ 現在ではLLMがそれっぽく考えてくれる

株式投資で考えられる投資戦略を10個挙げてください。



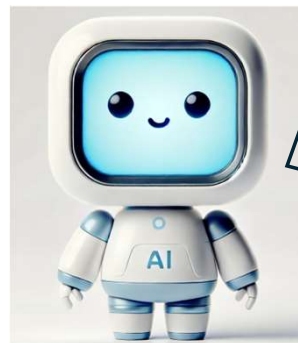
- 1. バリュート投資**
割安と判断される銘柄（低PER・低PBRなど）を選び、長期保有して企業価値の回復を待つ戦略。
- 2. グロース投資**
売上・利益の成長率が高い企業を選び、将来の成長期待で株価上昇を狙う戦略。
- 3. 配当株投資**
高配当利回りの銘柄を中心にポートフォリオを組み、安定的なインカムゲイン（配当）を追求する戦略。
- 4. モメンタム投資**
株価の上昇トレンドが続いている銘柄を買い、トレンドが終了したら売却するという、価格動向に追随する戦略。
- 5. インデックス投資（パッシブ運用）**
日経平均やTOPIX、海外の主要指数など市場全体に連動するETF・投資信託を買い、広く分散する戦略。
- 6. スマートベータ／ファクター投資**
価値（Value）、成長（Growth）、規模（Size）、低ボラティリティ（Low Volatility）など特定要因（ファクター）に着目して銘柄選定する戦略。
- 7. テーマ投資**
AI、環境（ESG）、DX、バイオテクノロジーなど、成長が見込まれるテーマ・セクターに関連する銘柄群へ集中投資する戦略。
- 8. ペアトレーディング**
同業他社ペアの株価差（スプレッド）が拡大／縮小するタイミングを狙い、「割高」銘柄を空売り、「割安」銘柄を買い、スプレッド回復で利ざやを獲得する戦略。
- 9. アービトラージ（裁定取引）**
上場市場や同一銘柄の異なる金融商品間で生じる価格差を同時売買で無リスクに稼ぐ戦略。
- 10. アルゴリズム／クオンツ運用**
数理モデルや統計的手法を用いて多量のデータを解析し、自動的に売買シグナルを生成・執行する戦略。

AIエージェントと投資戦略 2/2

(2) データありきで「投資戦略を考えて」

- ・ データサイエンスの世界と近しい
- ・ 特徴量エンジニアリング、特徴量開発
- ・ モデリング

J-Quants APIを使って翌日の株価を予測して収益を得るための投資戦略を10個挙げてください。



1. 単純移動平均 (SMA) クロスオーバー戦略

- ・ 概要：短期SMA（例：5日）と長期SMA（例：25日）のゴールデンクロス／デッドクロスで売買シグナルを生成。

2. 相対力指数 (RSI) 反転戦略

- ・ 概要：RSI が一定閾値（例：30以下の買い／70以上の売り）を跨いだタイミングでエントリー。

3. Fama-French 3ファクターモデルによる超過リターン予測

- ・ 概要：サイズ (SMB)、バリュー (HML) 因子を用いた回帰モデルで翌日の銘柄ごとの超過リターンを推定し、上位銘柄を買い／下位銘柄をショート。

4. 財務指標スクリーニング+回帰予測

- ・ 概要：PER・PBR・ROE・売上成長率などを説明変数に、多変量回帰（線形回帰 or LASSO）で翌日のリターンを予測。

5. ボラティリティブレイクアウト戦略

- ・ 概要：過去数日のATR (Average True Range) や実現ボラティリティを計算し、「ボラティリティ × k 倍」幅を超えたら順張りエントリー。

6. 機械学習 (LightGBM) による分類モデル

- ・ 概要：過去のテクニカル指標／ファンダメンタル指標を特徴量とし、翌日の株価上昇 (Yes/No) を二値分類。上昇確率上位から買い。

7. 時系列深層学習 (LSTM) による価格予測

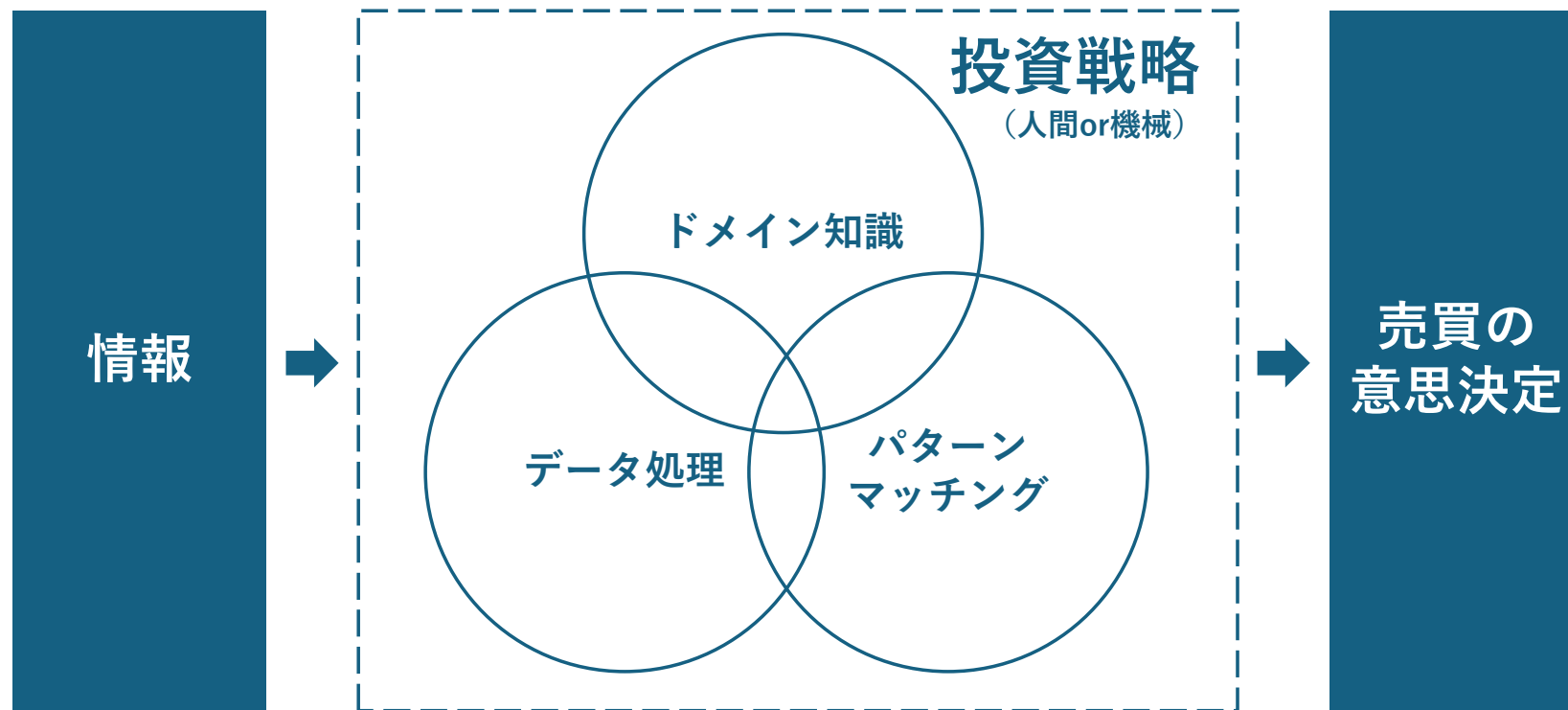
- ・ 概要：過去数週間～数ヶ月の株価系列を LSTM/GRU に入力し、翌日の終値レンジを回帰予測。

8. ESGスコア・スクリーニング戦略

- ・ 概要：ESG スコア上位銘柄を長期保有しつつ、翌日の短期リターンも併せて予測。ESG + テクニカル複合戦略。

本発表における投資戦略の定義

- ・ 手持ちの情報（データ）をもとに、売買を判断できる根拠（数値）を算出するプロセス



投資戦略を量産してどうする？

- ・ 多数の戦略を作り、パフォーマンスの最も良い戦略を運用する（1点張り）

→これは絶対にNG

- ・ 多数の戦略を作り、分散して運用する

→これは概ねOK

戦略間の相関に注意し、アンサンブルするなら均等ウェイトもしくは多数決モデルとする

投資戦略をどうやって量産するか？

帰納法（データドリブン）

- ・ 具体的な事例や観察から一般的な法則や結論を導き出す方法
- ・ データの意味は考えなくてOK。とにかく試行して、後から考える
- ・ データスヌーピングバイアスに注意（演繹的に除くか分散して対応する）

演繹法（セオリードリブン）

- ・ 一般的なルールや原則から具体的な結論を導き出す方法
- ・ 思考の枠組みに囚われやすく、戦略の量産には不向き

投資戦略量産の事例

(1) "Trader-Company法:メタヒューリスティクスを用いた株価予測", 伊藤, 2020
(2) "Generative Adversarial Networks for Financial Trading Strategies Fine-Tuning and Combination", Koshiyama, 2019
(3) "Automate Strategy Finding with LLM in Quant Investment", Kou, 2024

(1) Trader-Company法

- 二項演算、活性化関数、多項式の組み合わせによる予測器を多数作成する

$$\hat{r}_{i,t+p}^{(p)} = \sum_{i=1}^N w_i A_i(O_i(r_{P_i,t-D_i}, r_{Q_i,t-F_i})).$$

項数	VOD _{t+2} =
1	0.45 sin(PRU _t > KGF _{t-2})
2	-0.96(HSBA _{t-2} - RIO _{t-7}) - 0.91((GLEN _t > SKY _{t-1}))
3	-1.39 sin(GLEN _{t-7} - BT _{t-3}) - 0.94 sin(max(CRDA _{t-2} , CPI _{t-3})) - 0.34 tanh(GLEN _t > EXPN _{t-5})

(2) GAN for Financial Trading Strategies

- GANによって時系列サンプルを生成し、投資戦略のパラメータチューニングする
- 多数生成した戦略を弱学習器でアンサンブルする

(3) Automate Strategy Finding with LLM

- 論文やファイナンス記事からLLMがアルファの数式を抽出する
- マルチモーダルなデータと数式を組み合わせ、シャープで戦略選別する

帰納的

演繹的

投資戦略量産の手法一覧

手 法	概 要・メリット	デメリット
ランダム組み合わせ	・手持ちのデータやComponentをランダムに取捨選択して多数のパターンを作る（ランダムフォレスト等）	・組み合わせ同士の整合性を担保するため、機械的な設定になりがち
パラメータ探索	・ウィンドウを設定するだけで、簡単に戦略を水増しできる	・探索範囲が狭く、戦略の多様性に不足しがち
LLM/AIエージェント	・プロンプトを変更することで、従来の枠組みを超えた柔軟なアイデアを生成できる	・10～20個を超えると似通った提案が増えてくる ・Few-Shotで多様性を作る場合、結局コンテキストにランダム組み合わせが必要

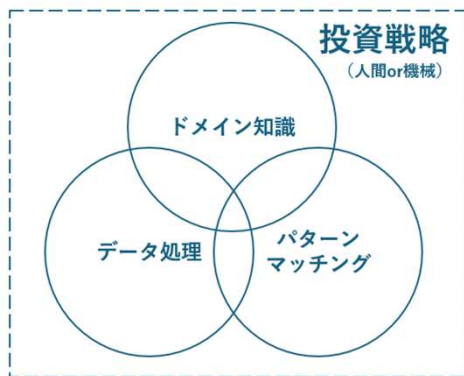
帰納的



演繹的

今回の方針

- ・ AIエージェント × ランダム組み合わせにより投資戦略を量産する
 - ・ 単純にAIエージェントに戦略生成させるだけでは、多様性が不足する
 - ・ ゆえに多様性を確保するためのフレームワークを人間が指定する必要がある



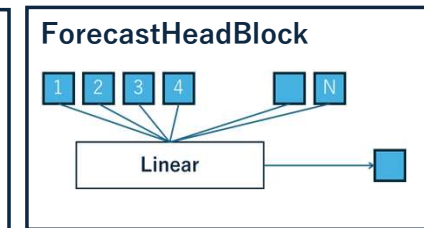
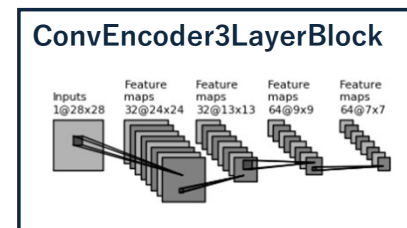
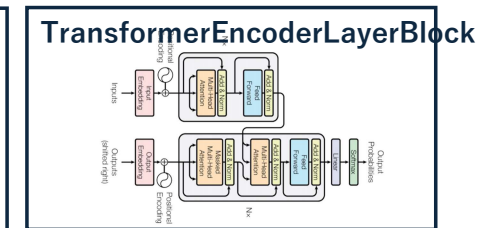
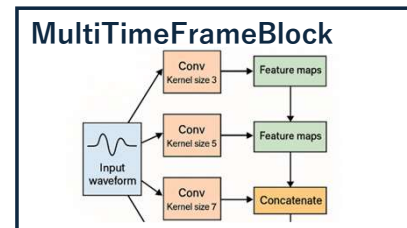
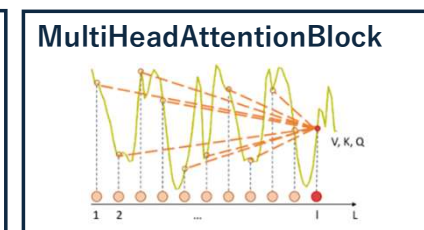
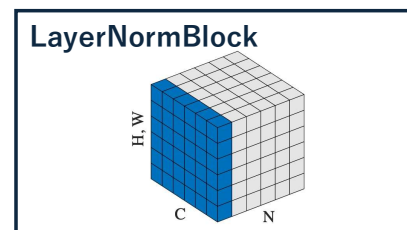
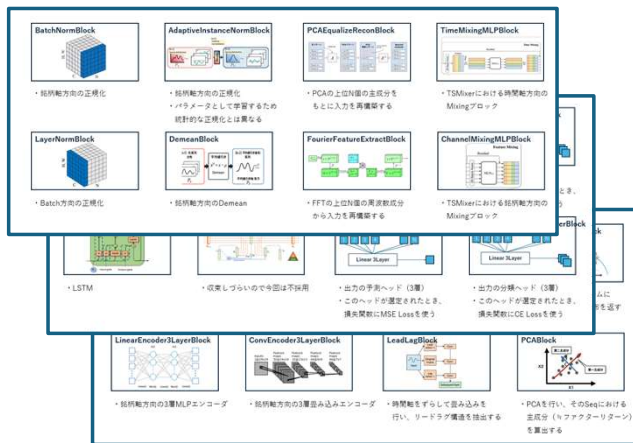
フレームワークの仕込みどころ

- (1) データ変換
- (2) プロンプティング
- (3) モデリング

今回は(1)と(3)を取りまとめて、End-to-Endなモデルをランダム量産する
LLMを使ってドメイン知識をコードに変換し、AIエージェントは量産の自動化を手助けする

今回提案：Alpha Architecture Agent

- ・ 予め時系列解析や金融のドメインブロックを作成しておく（人間+LLM）
- ・ AIエージェントは、これらのブロックを組み合わせてアーキテクチャ（戦略）を量産する

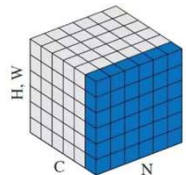


各ドメインブロックのコード生成

- ・ 基本的にChatGPTを使って生成する（ただし自分でもチェックして清書する）
 - (1)時系列ニューラルネットの論文のアーキテクチャをブロックに分割させる
 - (2)金融に効果のありそうなアーキテクチャを考案させる
 - (3)自身の持つドメイン情報を与え、それを表現するアーキテクチャを作成させる

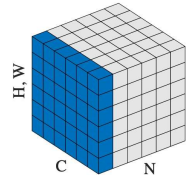
各ブロック紹介 1/7

BatchNormBlock



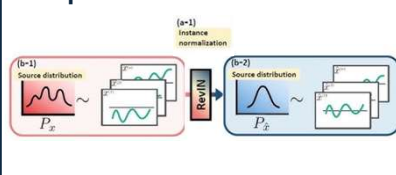
- ・ 銘柄軸方向の正規化

LayerNormBlock



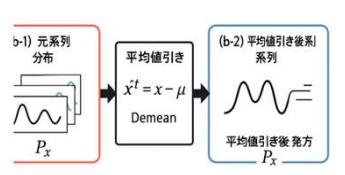
- ・ Batch方向の正規化

AdaptiveInstanceNormBlock



- ・ 銘柄軸方向の正規化
- ・ パラメータとして学習するため統計的な正規化とは異なる

DemmeanBlock



- ・ 銘柄軸方向のDemmean

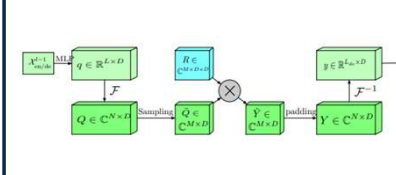
規格化ブロック

PCAEqualizeReconBlock



- ・ PCAの上位N個の主成分をもとに入力を再構築する

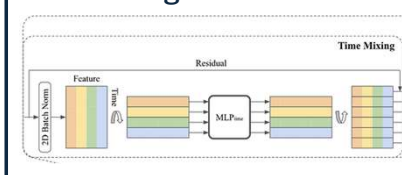
FourierFeatureExtractBlock



- ・ FFTの上位N個の周波数成分から入力を再構築する

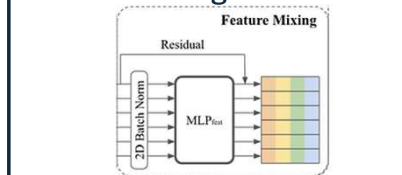
成分抽出ブロック

TimeMixingMLPBlock



- ・ TSMixerにおける時間軸方向のMixingブロック

ChannelMixingMLPBlock

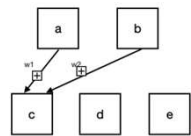


- ・ TSMixerにおける銘柄軸方向のMixingブロック

ミキシングブロック

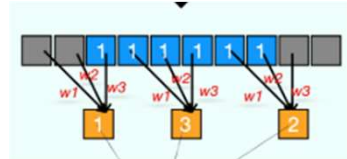
各ブロック紹介 2/7

LinearEncoderBlock



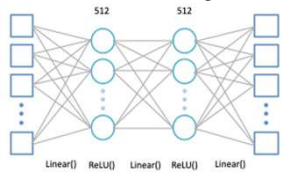
- ・ 銘柄軸方向の線形エンコーダ

ConvEncoderBlock



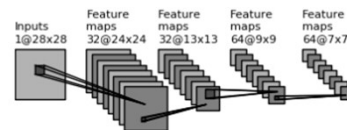
- ・ 銘柄軸方向の畳み込みエンコーダ

LinearEncoder3LayerBlock



- ・ 銘柄軸方向の3層MLPエンコーダ

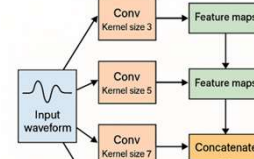
ConvEncoder3LayerBlock



- ・ 銘柄軸方向の3層畳み込みエンコーダ

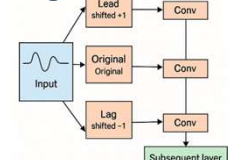
エンコーディングブロック

MultiTimeFrameBlock



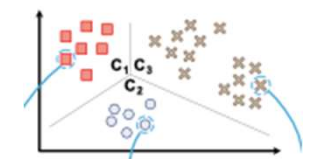
- ・ 複数のカーネルサイズで畳み込みを行い、マルチ時間軸の特徴を抽出する

LeadLagBlock



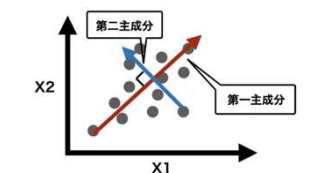
- ・ 時間軸をずらして畳み込みを行い、リードラグ構造を抽出する

RegimeDetectionBlock



- ・ Seq毎にN個のレジームに滞在している確率分布を返す

PCABlock

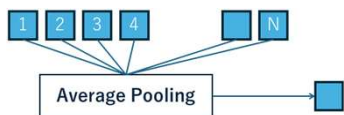


- ・ PCAを行い、そのSeqにおける主成分（≒ファクターリターン）を算出する

ファイナンスドメインブロック

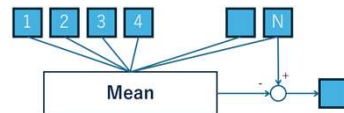
各ブロック紹介 3/7

FeatsAvgPoolingBlock



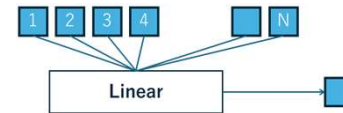
- ・特徴方向に平均プーリングして潰すブロック

FeatsMADevBlock



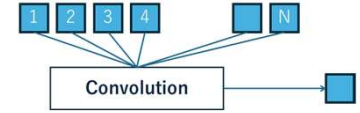
- ・特徴方向の特定の値の平均乖離を取得するブロック

FeatsLinearBlock



- ・特徴方向に線形層で潰すブロック

FeatsConvBlock



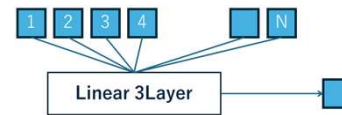
- ・特徴方向に線形畳み込みで潰すブロック

FeatsLatestBlock



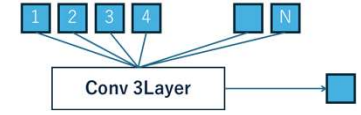
- ・特徴方向の特定の値を代表値として採用するブロック

FeatsLinear3LayerBlock



- ・特徴方向に3層MLPで潰すブロック

FeatsConv3LayerBlock

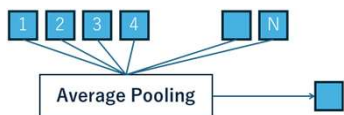


- ・特徴方向に3層畳み込みで潰すブロック

特徴方向を統合するブロック

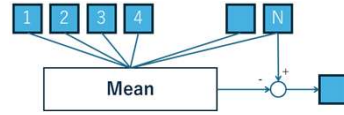
各ブロック紹介 4/7

SeqAvgPoolingBlock



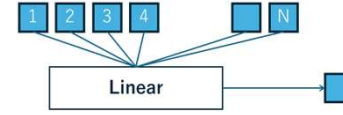
- ・ 時間軸方向に平均プーリングして潰すブロック

SeqMADevBlock



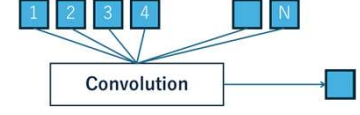
- ・ 時間軸方向の最終Seqの値の平均乖離を取得するブロック

SeqLinearBlock



- ・ 時間軸方向に線形層で潰すブロック

SeqConvBlock



- ・ 時間軸方向に線形畳み込みで潰すブロック

SeqLatestBlock



- ・ 時間軸方向の最終Seqの値を代表値として採用するブロック

SeqLinear3LayerBlock



- ・ 時間軸方向に3層MLPで潰すブロック

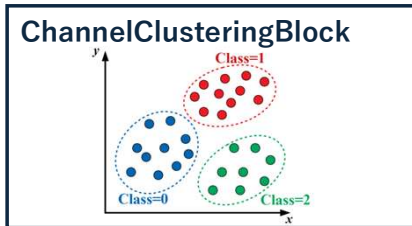
SeqConv3LayerBlock



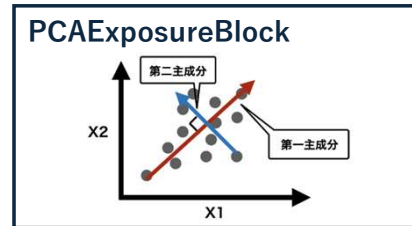
- ・ 時間軸方向に3層畳み込みで潰すブロック

時間軸方向を統合するブロック

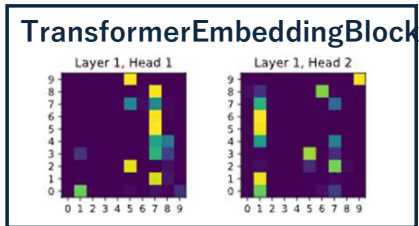
各ブロック紹介 5/7



- ・ 銘柄毎にN個のクラスタに存在している確率分布を返す



- ・ 各銘柄の主成分エクスポージャーを算出する

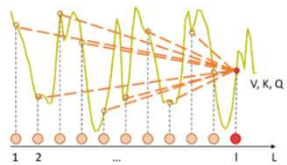


- ・ 銘柄間の相互関係を抽出する

銘柄毎の特徴付けブロック

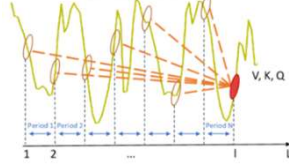
各ブロック紹介 6/7

MultiHeadAttentionBlock



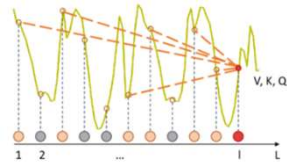
- 一般的なマルチヘッドアテンション

AutoCorrelationBlock



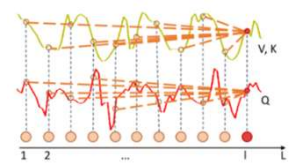
- Autoformerに採用されている自己相関ベースのアテンションブロック

ProbSparseSelfAttentionBlock



- Informerに採用されているスパースなアテンションブロック

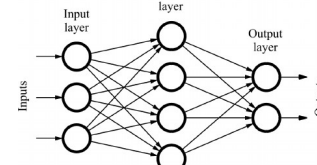
CrossSelfAttentionBlock



- Crossformerに採用されているCross-Windowのセルフアテンションブロック

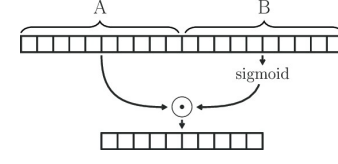
Attentionブロック (Transformerを使う場合)

FeedForwardBlock



- 通常の2層MLPのFeedForward層

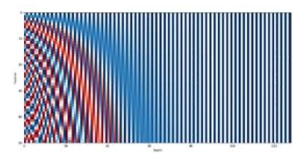
GatedFeedForward Block



- Gateを使ったFeedForward層

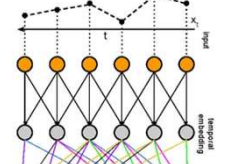
FFブロック (Transformerを使う場合)

PositionalEncodingBlock



- 位置エンコーディング

TimeEmbeddingBlock

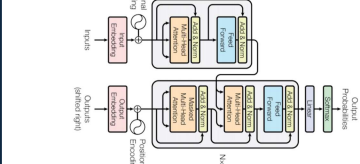


- 時刻エンベディング

時間埋め込みブロック

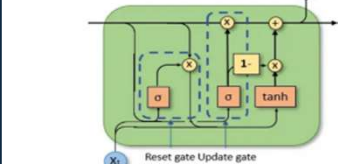
各ブロック紹介 7/7

TransformerEncoderLayerBlock



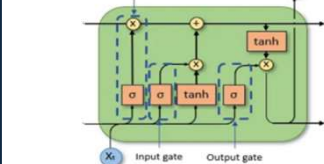
- Transformerだが、アテンションとFF層は前頁のブロックで組み替えられる

GRUBlock



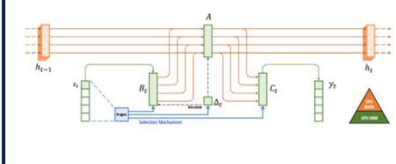
- Gate Recurrent Unit

LSTMBlock



- LSTM

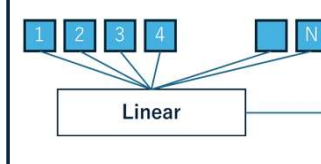
MambaBlock



- 収束しづらいので今回は不採用

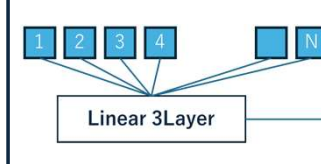
特徴抽出ブロック

ForecastHeadBlock



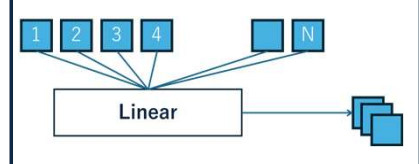
- 出力の予測ヘッド
- このヘッドが選定されたとき、損失関数にMSE Lossを使う

ForecastHead3LayerBlock



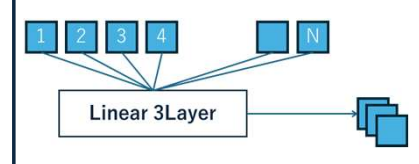
- 出力の予測ヘッド（3層）
- このヘッドが選定されたとき、損失関数にMSE Lossを使う

ForecastHeadClassBlock



- 出力の分類ヘッド
- このヘッドが選定されたとき、損失関数にCE Lossを使う

ForecastHeadClass3LayerBlock



- 出力の分類ヘッド（3層）
- このヘッドが選定されたとき、損失関数にCE Lossを使う

予測ヘッドブロック

プロンプト紹介

- ・ ブロックのコードは与えておく
- ・ AIエージェントはブロックを選定し、shapeの整合性を考慮して全体のコードを作る

```
instructions = """
以下の要領で、金融時系列を予測するためのニューラルネットアーキテクチャのコードを作成してください。

# アーキテクチャの入出力のshape
- 入力: input_dim, seq_len)です。
- 出力: output_dim)です。
- ここで、input_dim=200, seq_len=252, output_dim=200です。

# ニューラルネットアーキテクチャのコード作成要領
## 全体
- ニューラルネットアーキテクチャのコードは、予めコードが与えられているアーキテクチャブロックを組み合わせて作成します。
- 各アーキテクチャブロックの入出力のshapeの次元の並びは、(B, dim, len)を基本とします。
- 各ブロックへ接続する際、前段のブロックの出力のshapeと後段のブロックの入力のshapeが同じとなるよう、注意してパラメータを設定してください。
- 必ず、コメントでそのブロックの出力のshapeを明示するようにしてください。

## 各ブロックの選定
- データの正規化に使うブロックを、[BatchNormBlock, LayerNormBlock, LayerNorm2DBlock, DemeanBlock, AdaptiveInstanceNormBlock]から1つ選定
- 入力データの変換に使うブロックを、[PCAEEqualizeReconstructBlock, FourierFeatureExtractorBlock, TimeMixingMLPBlock, ChannelMixingMLPBlock]
- 位置や時間に関するエンコーディングを、[PositionalEncodingBlock, TimeEmbeddingBlock, None]から1つ選定してください。
- Seq方向の情報を取りまとめた特徴量作成するブロックを、[SeqAvgPoolingBlock, SeqLatestBlock, SeqMADevBlock, SeqLinearBlock, SeqLinear3L]
- 各銘柄固有の特徴を作成するブロックを、[ChannelClusteringBlock, TransformerEmbeddingBlock, PCAExposureBlock, None]から1つ選定してください。
- 銘柄間の情報を元にそのSeqのレジームを算出するブロックを、[PCABlock, LinearEncoderBlock, LinearEncoder3LayerBlock, ConvEncoderBlock, ]
- そのSeqのレジームを元に、各銘柄の動向を算出するブロックを、[FeatsAvgPoolingBlock, FeatsLatestBlock, FeatsMADevBlock, FeatsLinearBlock]
- Transformerを使う場合、Attentionブロックを、[MultiHeadAttentionBlock, ProbSparseSelfAttentionBlock, AutoCorrelationBlock, LocalSelfA]
- Transformerを使う場合、FeedForwardブロックを、[FeedForwardBlock, GatedFeedForwardBlock]から1つ選定してください。
- 出力の予測ヘッドブロックを、[ForecastHeadBlock, ForecastHead3LayerBlock]から1つ選定してください。

# アーキテクチャブロックの次元に関連する変数名の説明
- B: バッチ数。
- input_dim: 予測対象の銘柄数であり、今回はinput_dim=200です。
- model_dim: ニューラルネットアーキテクチャ内での変量の次元です。input_dimと同じ場合もあれば異なる場合もあります。また各ブロックにおいて
- hidden_dim: ブロック内での一時的な変量の次元です。
- emb_dim: embeddingされた次元です。

```

```
architectures = """
1. Normarization

class BatchNormBlock(nn.Module):
    # 説明 : 入力チャネル次元をバッチ正規化する最小単位ブロック。
    # 入力 : (B, D) もしくは (B, D, L)
    # 出力 : (B, D) もしくは (B, D, L)
    def __init__(self, d_model: int):
        super().__init__()
        self.bn = nn.BatchNorm1d(d_model)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.bn(x)

class LayerNormBlock(nn.Module):
    # 説明 : 入力チャネル次元をレイヤー正規化するブロック。
    # 入力 : (B, L, D) もしくは (B, D)
    # 出力 : (B, L, D) もしくは (B, D)
    def __init__(self, d_model: int):
        super().__init__()
        self.ln = nn.LayerNorm(d_model)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.ln(x)

class DemeanBlock(nn.Module):
    # 説明 : 指定次元dimに沿って平均を引き、ゼロ中心化するブロック。
    # 入力 : (B, D, L)
    # 出力 : (B, D, L)
    def __init__(self, dim: int):
        super().__init__()

```

フレームワーク

- ・ 対象は日本株、デイリーのロングショート
- ・ Close to Closeを時系列で予測する
- ・ 入力はshape=(Batch, num_stocks=200, seq_len=252)のリターン系列
- ・ 5分位して上位1分位（40銘柄）ロング、下位1分位（40銘柄）ショート
- ・ 学習期間は2017年～2023年、検証期間は2024年～2025年2月末まで
- ・ 検証期間のシャープレシオでランキングする

結果：Leaderboard

- ・ 検証期間（2024年～2025年2月末）におけるパフォーマンス
- ・ 戦略総数70個しかないため、以下は参考データ

RANK	NAME	SHARPE	PARAMS
1	LayerNorm-None-None-SeqAvgPooling-ChannelClustering-PCA-FeatsConv-ProbSparseSelfAttention-FeedForward-ForecastHeadClass3Layer	1.3899	2601
2	AdaptiveInstanceNorm-None-PositionalEncoding-SeqMADev-PCAExposure-CalcCounter-FeatsConv3Layer-AutoCorrelation-GatedFeedForward-ForecastHeadClass3Layer	1.2573	1487
3	AdaptiveInstanceNorm-TimeMixingMLP-PositionalEncoding-SeqConv3Layer-PCAExposure-ConvEncoder3Layer-FeatsLinear-CrossSelfAttention-GatedFeedForward-ForecastHead	1.2353	8892
4	BatchNorm-None-PositionalEncoding-SeqLatest-ChannelClustering-LeadLag-FeatsConv3Layer-AutoCorrelation-FeedForward-ForecastHead	0.8720	5829
5	Demean-None-None-SeqMADev-TransformerEmbedding-CalcStd-FeatsLinear3Layer-AutoCorrelation-FeedForward-ForecastHead	0.8184	260078
6	LayerNorm2D-FourierFeatureExtractor-None-SeqLatest-ChannelClustering-ConvEncoder-FeatsConv3Layer-LocalSelfAttention-GatedFeedForward-ForecastHead3Layer	0.7765	103603
7	LayerNorm2D-TimeMixingMLP-PositionalEncoding-SeqLinear3Layer-None-MultiTimeFrame-FeatsMADev-CrossSelfAttention-FeedForward-ForecastHead3Layer	0.7015	115075
8	LayerNorm-None-None-SeqAvgPooling-ChannelClustering-ConvEncoder3Layer-FeatsConv3Layer-LocalSelfAttention-FeedForward-ForecastHeadClass	0.6879	2067
9	Demean-None-PositionalEncoding-SeqLinear-PCAExposure-LinearEncoder-FeatsMADev-LocalSelfAttention-FeedForward-ForecastHead	0.6495	2291
10	Demean-ChannelMixingMLP-None-SeqLatest-PCAExposure-LeadLag-FeatsMADev-LocalSelfAttention-GatedFeedForward-ForecastHead3Layer	0.6235	10146
11	AdaptiveInstanceNorm-ChannelMixingMLP-None-SeqLinear-None-CalcCounter-FeatsConv-LocalSelfAttention-GatedFeedForward-ForecastHead3Layer	0.5198	7532
12	BatchNorm-None-None-SeqLatest-TransformerEmbedding-LeadLag-FeatsLinear3Layer-LocalSelfAttention-FeedForward-ForecastHeadClass	0.4460	265565
13	LayerNorm2D-PCAEqualizeReconstruct-PositionalEncoding-SeqMADev-ChannelClustering-LinearEncoder3Layer-FeatsLinear-LocalSelfAttention-GatedFeedForward-ForecastHead3Layer	0.4407	105283
14	LayerNorm-TimeMixingMLP-None-SeqLinear3Layer-None-RegimeDetection-FeatsConv3Layer-LocalSelfAttention-GatedFeedForward-ForecastHead	0.4404	32266
15	AdaptiveInstanceNorm-None-PositionalEncoding-SeqConv3Layer-PCAExposure-LSTM-FeatsLatest-AutoCorrelation-FeedForward-ForecastHead	0.4278	4876
16	LayerNorm2D-ChannelMixingMLP-PositionalEncoding-SeqAvgPooling-ChannelClustering-LeadLag-FeatsLatest-AutoCorrelation-FeedForward-ForecastHead	0.4217	110282
17	AdaptiveInstanceNorm-TimeMixingMLP-TimeEmbedding-SeqAvgPooling-TransformerEmbedding-LSTM-FeatsAvgPooling-AutoCorrelation-FeedForward-ForecastHead3Layer	0.3860	268833
18	AdaptiveInstanceNorm-TimeMixingMLP-None-SeqAvgPooling-TransformerEmbedding-CalcCounter-FeatsLatest-CrossSelfAttention-GatedFeedForward-ForecastHeadClass	0.2748	263574
19	LayerNorm-PCAEqualizeReconstruct-PositionalEncoding-SeqLinear-ChannelClustering-ConvEncoder3Layer-FeatsLinear3Layer-AutoCorrelation-FeedForward-ForecastHeadClass	0.2682	6848
20	LayerNorm-FourierFeatureExtractor-None-SeqLatest-TransformerEmbedding-ConvEncoder3Layer-FeatsConv3Layer-MultiHeadAttention-FeedForward-ForecastHead3Layer	0.2461	259905

結果：効果のあるブロックランキング

- ・ そのブロックが含まれている群と含まれていない群のシャープの平均差分でランキング
- ・ 参考データ（COUNTが不足、SRを差分比較してよいか議論）

BEST BLOCK

RANK	BLOCK	COUNT	SR_DIFF
1	FeatsConv3LayerBlock	9	0.5378
2	SeqAvgPoolingBlock	9	0.3863
3	FeatsConvBlock	17	0.3727
4	ForecastHead3LayerBlock	16	0.3700
5	PCAExposureBlock	17	0.3345

WORST BLOCK

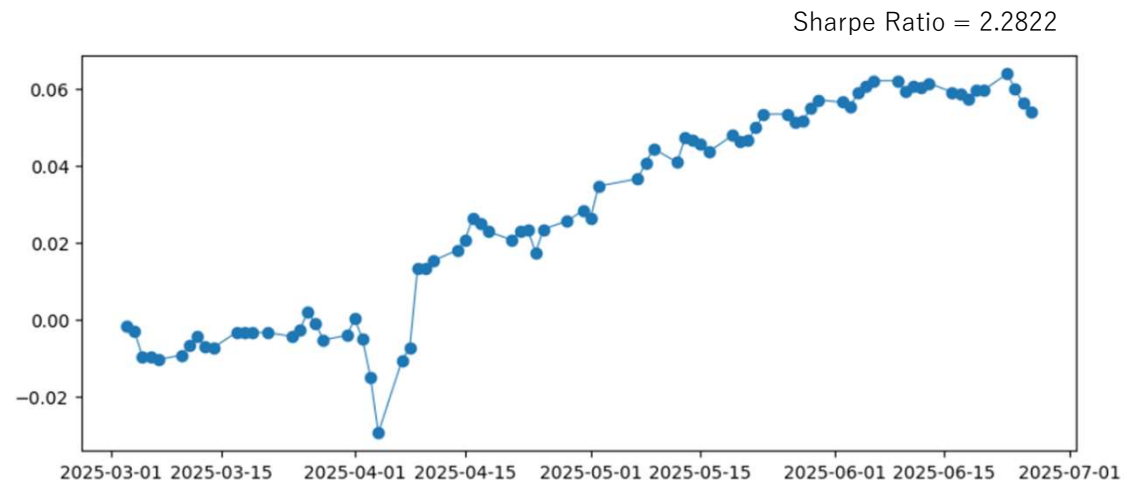
RANK	BLOCK	COUNT	SR_DIFF
41	SeqLinear3LayerBlock	14	-0.3357
42	PCAEqualizeReconstructBlock	12	-0.3396
43	CalcMeanBlock	7	-0.6221
44	TransformerEncoderLayerBlock	2	-0.6999
45	LinearEncoder3LayerBlock	3	-0.7664

結果：アンサンブルポートフォリオ

- ・ 完全なフォワード期間（2025年3月～2025年6月末）のパフォーマンス

- ・ 上位20戦略を均等ウェイトアンサンブル $ensemble\ pred = rank\left(\sum_{i=1}^{20} rank(pred_i)\right)$

RANK	NAME	SHARPE	FW_SR
1	LayerNorm-None-None-SeqAvgPooling-ChannelClu	1.3899	1.0682
2	AdaptiveInstanceNorm-None-PositionalEncoding-S	1.2573	-0.7531
3	AdaptiveInstanceNorm-TimeMixingMLP-PositionalE	1.2353	1.9780
4	BatchNorm-None-PositionalEncoding-SeqLatest-Cl	0.8720	0.4696
5	Demean-None-None-SeqMADev-TransformerEmbe	0.8184	-0.0540
6	LayerNorm2D-FourierFeatureExtractor-None-SeqL	0.7765	0.9321
7	LayerNorm2D-TimeMixingMLP-PositionalEncoding	0.7015	3.9765
8	LayerNorm-None-None-SeqAvgPooling-ChannelClu	0.6879	1.3716
9	Demean-None-PositionalEncoding-SeqLinear-PCAF	0.6495	-0.8172
10	Demean-ChannelMixingMLP-None-SeqLatest-PCAF	0.6235	4.3224
11	AdaptiveInstanceNorm-ChannelMixingMLP-None-S	0.5198	4.6233
12	BatchNorm-None-None-SeqLatest-TransformerErr	0.4460	0.2053
13	LayerNorm2D-PCAEqualizeReconstruct-PositionalE	0.4407	4.8379
14	LayerNorm-TimeMixingMLP-None-SeqLinear3Laye	0.4404	-0.0278
15	AdaptiveInstanceNorm-None-PositionalEncoding-S	0.4278	-2.6583
16	LayerNorm2D-ChannelMixingMLP-PositionalEncod	0.4217	1.7538
17	AdaptiveInstanceNorm-TimeMixingMLP-TimeEmbe	0.3860	4.9894
18	AdaptiveInstanceNorm-TimeMixingMLP-None-Seq	0.2748	0.0068
19	LayerNorm-PCAEqualizeReconstruct-PositionalEnc	0.2682	2.3268
20	LayerNorm-FourierFeatureExtractor-None-SeqLat	0.2461	0.8349



まとめ

- ・正直、頓挫すると思って企画を進めていたが、予想以上に良い結果が得られた
(知見としてもパフォーマンスとしても)
- ・印象として、やはりニューラルネットは強い
当然フィッティングしているものもあるが、OOSで予測性能が残っているモデルは何かしら堅牢な特徴を捉えているイメージ
- ・今回の発表はボリュームが多くなってしまったので、
裏話的な雑談を後日自分のチャンネルで配信します！